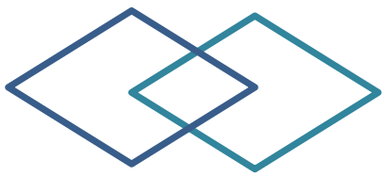


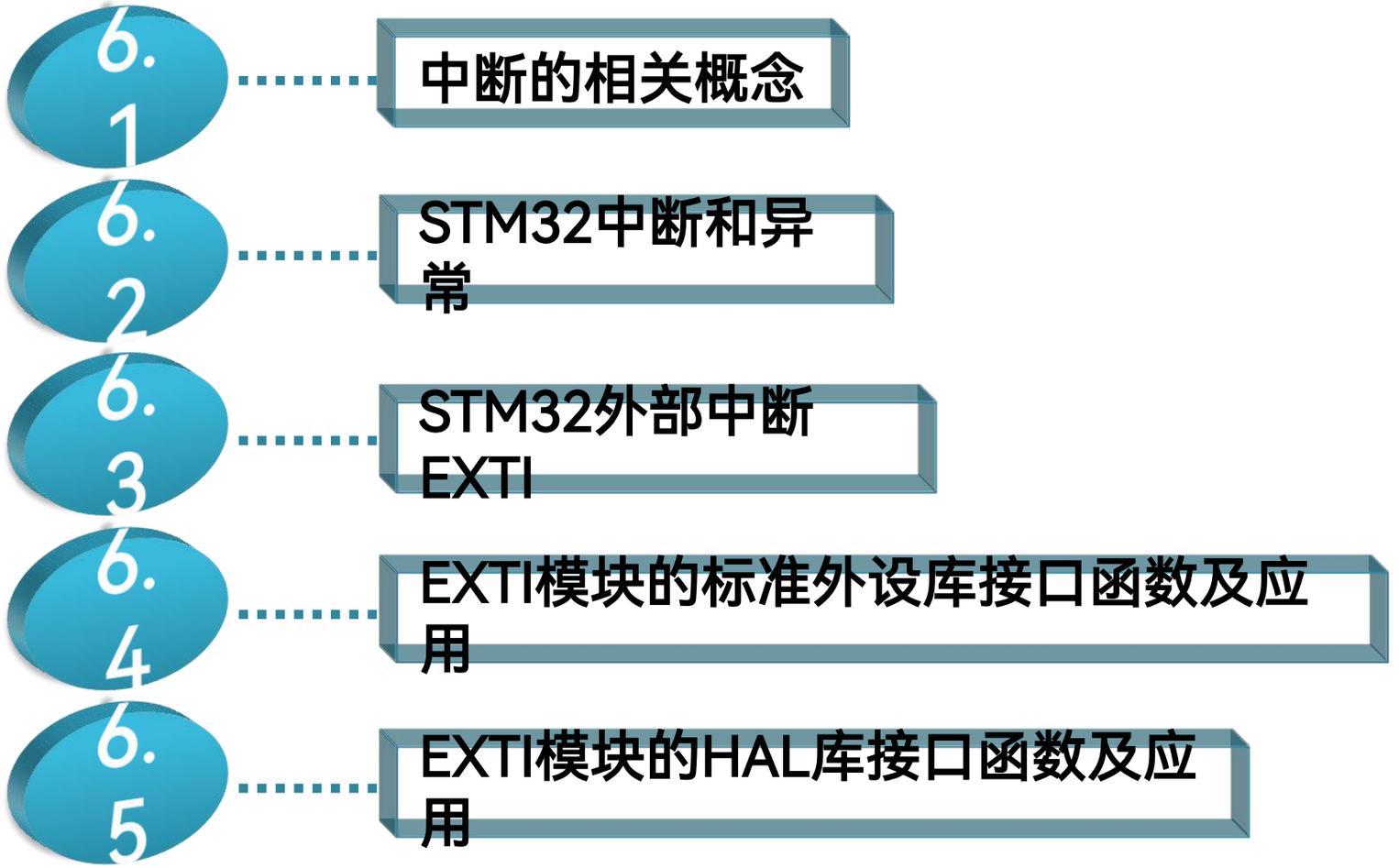
第6章 中 断



本章知识与能力要求

- 理解和掌握中断的概念和中断处理过程；
- 理解和掌握STM32F103微控制器的中断类型、优先级概念和中断向量表；
- 了解STM32F103微控制器的NVIC中断结构和特点；
- 掌握STM32F103微控制器EXTI的内部结构、工作原理和特性；
- 熟悉NVIC和EXTI相关的标准外设库函数和HAL库函数；
- 掌握STM32的中断配置过程；
- 掌握基于标准外设库函数进行外部中断的开发过程；
- 掌握基于HAL库进行外部中断的开发过程。

第6章 中断


6.1
中断的相关概念
6.2
STM32中断和异常
6.3
STM32外部中断EXTI
6.4
EXTI模块的标准外设库接口函数及应用
6.5
EXTI模块的HAL库接口函数及应用

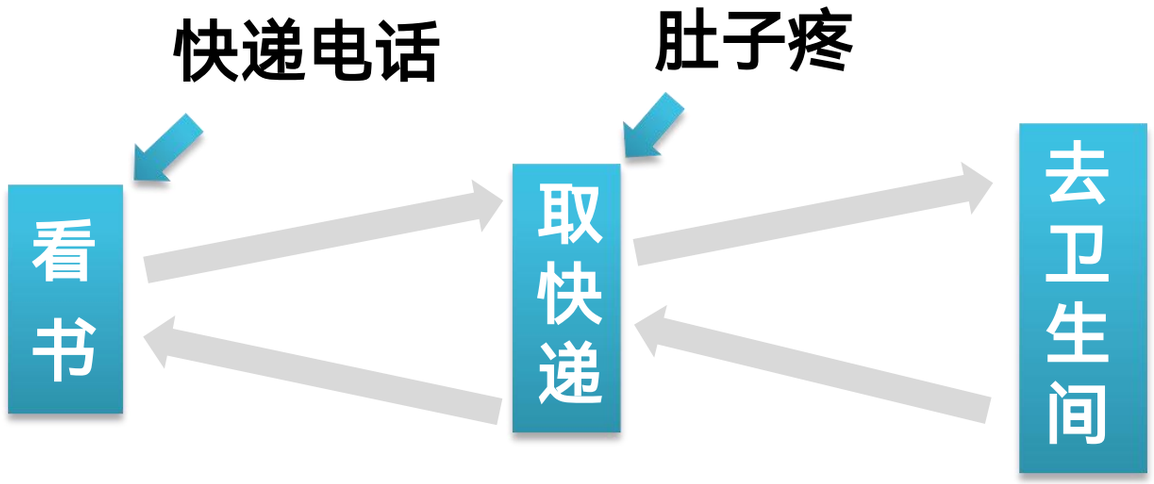
6.1 中断的相关概念

1 6.1.1 什么是中断?

2 6.1.2 为什么使用中断?

3 6.1.3 中断处理流程

6.1.1 什么是中断?



看书

快递电话、肚子疼

取快递、去卫生间

主程序

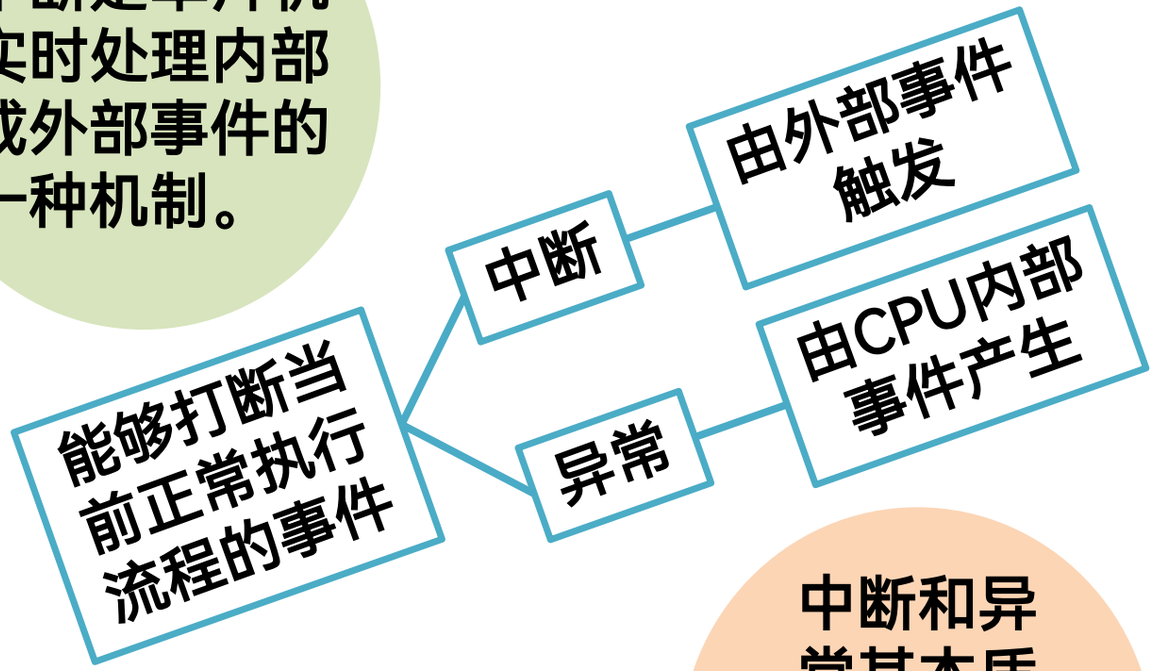
中断源

中断服务程序

6.1.1 什么是中断?

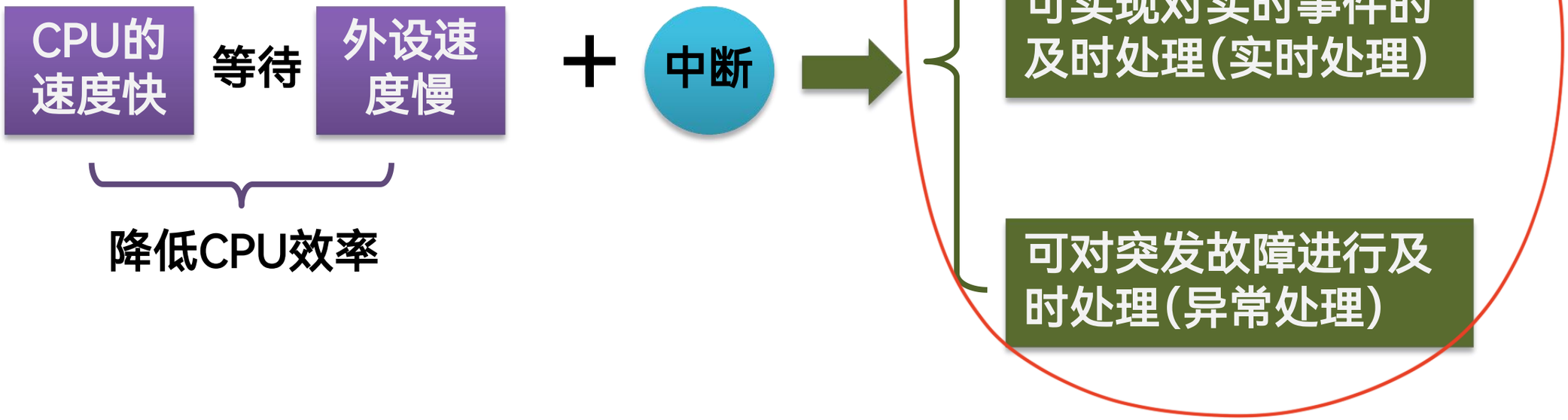
计算机在执行程序过程中，当出现**异常**情况(断电等)或特殊**请求**(数据传输等)时，计算机暂停现行程序的运行，转向对这些异常情况或特殊请求进行处理，处理完毕后再返回到现行程序的中断处，继续执行原程序，这就是“中断”。

中断是单片机实时处理内部或外部事件的一种机制。



中断和异常其本质都是对主程序的“中断”

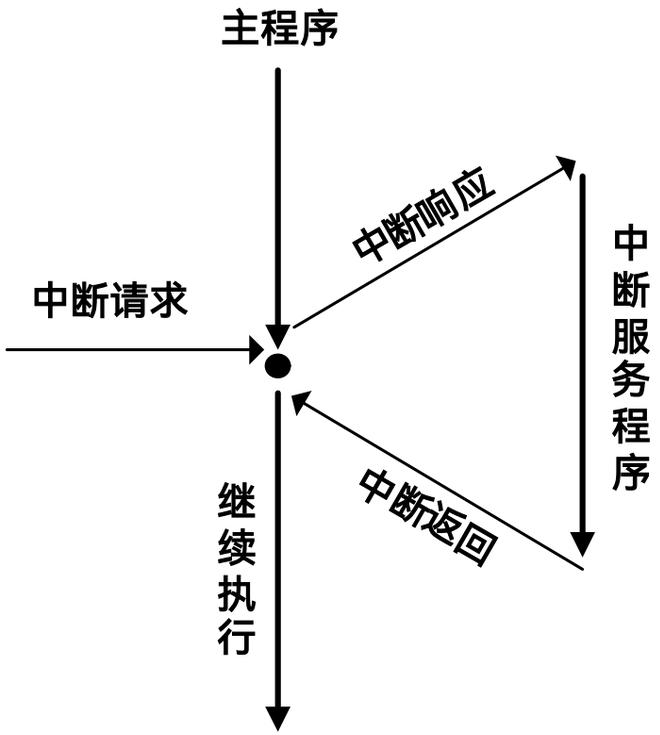
6.1.2 为什么使用中断?



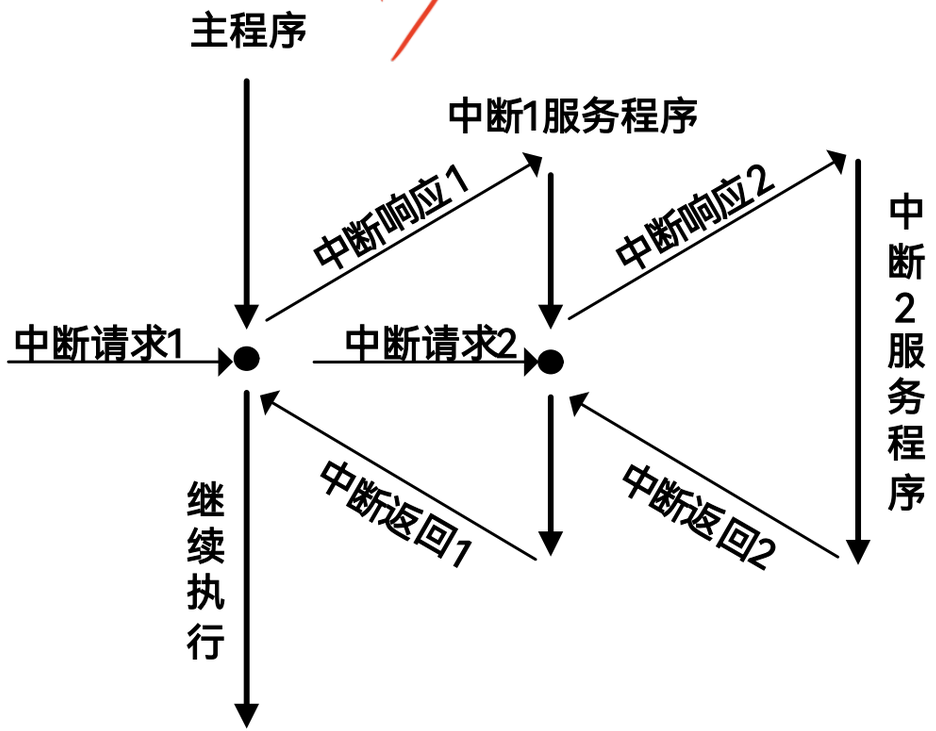
中断在嵌入式系统占有极其重要的地位，中断机制使得系统能更有效更合理地发挥效能和提高效率。

6.1.3 中断处理流程

中断处理流程：中断请求、中断响应、中断服务和中断返回



单重中断的中断处理流程



多重中断的中断处理流程

6.1.3 中断处理流程

中断请求

中断请求是中断源向CPU发出中断请求信号，此时中断控制系统的中断请求寄存器被置位，向CPU请求中断。

中断响应

CPU的中断系统判断中断源的中断请求是否符合中断响应条件，如果符合条件，则暂时中断当前程序并控制程序跳转到中断服务程序

中断服务

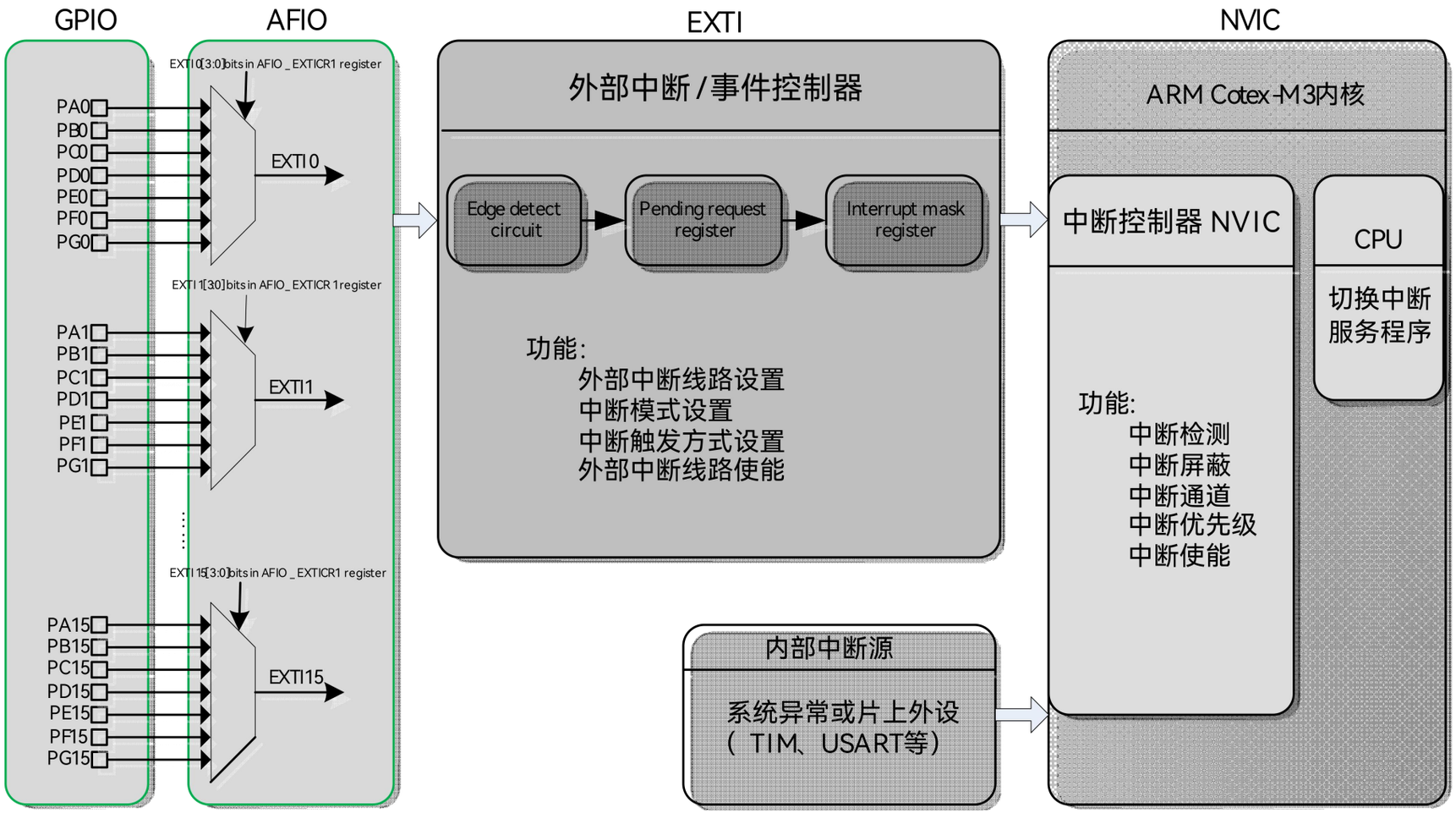
为处理中断而编写的程序称为中断服务程序，是由开发人员针对具体中断所要实现的功能进行设计和编写的，需要由开发人员来实现。

中断返回

CPU退出中断服务程序，返回到中断请求响应之前被中止的位置继续执行主程序。这部分操作同样由硬件来实现，不需要开发人员进行处理。

6.2 STM32中断和异常

STM32的内部中断处理机制



6.2 STM32中断和异常

6.2.1 STM32中断和异常向量表

6.2.2 STM32中断优先级

6.3.3 STM32中断服务程序

6.2.1 STM32中断和异常向量表

ARM公司设计的Cortex-M3内核可支持256个中断：

- 15个内核中断
- 240个外部中断

具有256级的可编程中断优先级设置，即除了3个固定的高优先级(Reset、NMI、硬件失效），其他中断和异常的优先级是可以由用户进行设置的

使用Cortex-M3内核的芯片制造商对中断进行精简

- STM32F10x系列产品有84个中断通道，包括16个内核中断和68个可屏蔽中断；
- STM32F103系列芯片只有60个可屏蔽中断
- STM32F107系列有68个可屏蔽中断。

6.2.1 STM32中断和异常向量表

中断向量表

当发生了异常或中断，内核要想响应这些异常或中断，就需要知道这些异常或中断的服务程序的**入口地址**，再由入口地址找到相应的中断服务程序，由中断入口地址组成的表称作中断向量表。

入口地址一般存放在程序存储器（ROM），默认情况下，Cortex-M3内核的中断向量表从零地址处开始，且每个向量占用4个字节。

向量编号	向量入口地址	说明
-	0x00000000	MSP的初始值
1	0x00000004	复位向量(PC初始值)
2	0x00000008	NMI异常服务程序的入口地址
3	0x0000000C	硬Fault异常服务程序的入口地址
…	…	其它中断服务程序的入口地址

6.2.1 STM32中断和异常向量表

STM32在标准外设库stm32f10x.h文件中，通过宏定义将中断通道号(中断号）与宏名联系起来，这样在使用中断时，直接引用宏名即可。

```
typedef enum IRQn
{
        /*Cortex-M3 Processor Exceptions Numbers */
  NonMaskableInt_IRQn= -14,
         /*!< 2 Non Maskable Interrupt  */
  MemoryManagement_IRQn= -12,
        /*!< 4 Cortex-M3 Memory Management
Interrupt */
  BusFault_IRQn = -11,
        /*!< 5 Cortex-M3 Bus Fault Interrupt */
……
} IRQn_Type;
```

6.2.2 STM32中断优先级

STM32使用Cortex-M3的8位优先级寄存器中的4位来配置中断优先级，即STM32中的NVIC **只支持16级中断优先级**的管理。

NVIC中断配置的相关函数存放在标准外设库misc.c和misc.h文件中，共定义了5个相关函数及NVIC初始化结构体

```
void NVIC_PriorityGroupConfig(uint32_t NVIC_PriorityGroup);
void NVIC_Init(NVIC_InitTypeDef* NVIC_InitStruct);
void NVIC_SetVectorTable(uint32_t NVIC_VectTab, uint32_t Offset);
void NVIC_SystemLPConfig(uint8_t LowPowerMode, FunctionalState NewState);
void SysTick_CLKSourceConfig(uint32_t SysTick_CLKSource);
```

6.2.2 STM32中断优先级

中断分组管理函数

void NVIC_PriorityGroupConfig(uint32_t NVIC_PriorityGroup)

用于设置中断的**优先级分组**，此函数只有一个参数NVIC_PriorityGroup，其取值共有5组，每组的抢占优先级和响应优先级所占位数均不同，取值范围不同

NVIC_PriorityGroup	抢占优先级取值范围	响应优先级取值范围	描述
NVIC_PriorityGroup_0	0	0~15	抢占优先级占0位，响应优先级占4位
NVIC_PriorityGroup_1	0，1	0~7	抢占优先级占1位，响应优先级占3位
NVIC_PriorityGroup_2	0，1，2，3	0，1，2，3	抢占优先级占2位，响应优先级占2位
NVIC_PriorityGroup_3	0，1，2，3，4，5，6，7	0，1	抢占优先级占3位，响应优先级占1位
NVIC_PriorityGroup_4	0~15	0	抢占优先级占4位，响应优先级占0位

6.2.2 STM32中断优先级

结构体

中断初始化函数: void NVIC_Init(NVIC_InitTypeDef * NVIC_InitStruct)

作用：设置抢占优先级和响应优先级

指向NVIC_InitTypeDef结构体的指针

```
typedef struct
{
  uint8_t  NVIC_IRQChannel;//配置中断源，IRQ通道
  uint8_t  NVIC_IRQChannelPreemptionPriority; //配置抢占优先级
  uint8_t  NVIC_IRQChannelSubPriority; //配置响应优先级
  FunctionalState  NVIC_IRQChannelCmd; //使能中断通道
} NVIC_InitTypeDef;
```

6.2.2 STM32中断优先级

中断优先级判断原则

高抢先优先级的中断可以打断低抢先优先级的中断服务，构成中断嵌套；

抢占优先级的优先级总是高于响应优先级；

中断优先级的数值越小，优先级级别越高；



Reset NMI Hard Fault的优先级为负，且不可修改，高于普通的中断优先级。

中断优先级判断：先判断抢占优先级的大小，如果抢占优先级相同，则比较响应优先级的大小，若抢占优先级和响应优先级均相同，则根据中断向量表中的顺序来决定；

6.2.3 STM32中断服务程序

STM32将中断服务程序统一放在标准外设库stm32f10x_it.c文件中，其中的每个中断服务函数都只有函数名，函数体都是空的，需要用户自己编写相应的函数体，但中断服务程序的函数名是不能更改的

stm32f10x_it.c

```
/* Includes ------------------------------------------------------------------*/
#include "stm32f10x_it.h"

/** @addtogroup STM32F10x_StdPeriph_Template
  * @{
  */

/* Private typedef -----------------------------------------------------------*/
/* Private define ------------------------------------------------------------*/
/* Private macro -------------------------------------------------------------*/
/* Private variables ---------------------------------------------------------*/
/* Private function prototypes -----------------------------------------------*/
/* Private functions ---------------------------------------------------------*/

/******************************************************************************/
/*            Cortex-M3 Processor Exceptions Handlers                         */
/******************************************************************************/

/**
  * @brief  This function handles NMI exception.
  * @param  None
  * @retval None
  */
void NMI_Handler(void)
{
}

/**
  * @brief  This function handles Hard Fault exception.
  * @param  None
  * @retval None
  */
void HardFault_Handler(void)
{
  /* Go to infinite loop when Hard Fault exception occurs */
  while (1)
  {
  }
}

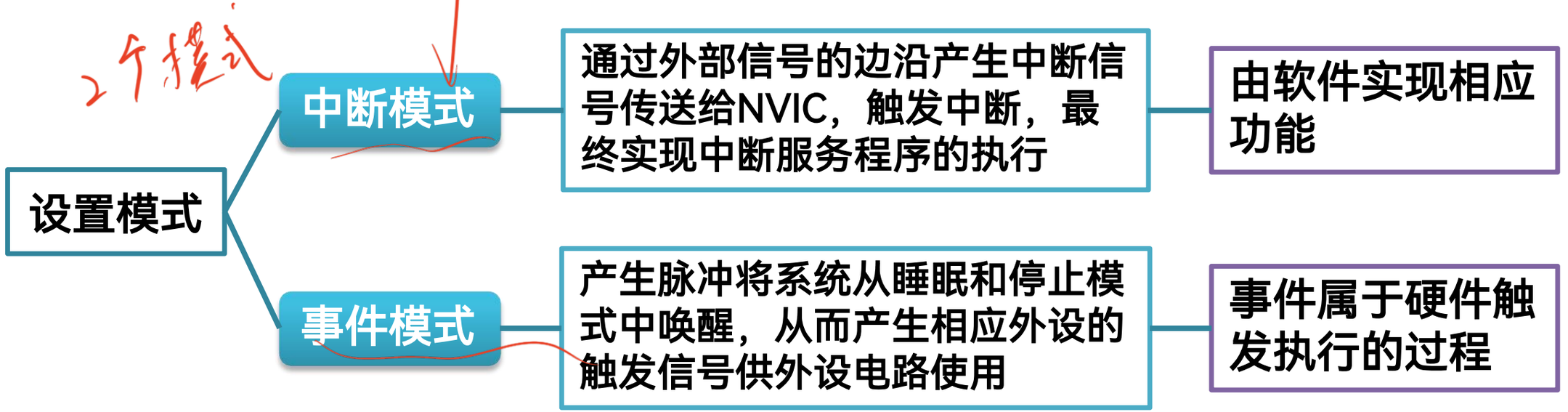
/**
  * @brief  This function handles Memory Manage exception.
  * @param  None
```

stm32f10x_it.c中断处理函数

6.3 STM32外部中断EXTI

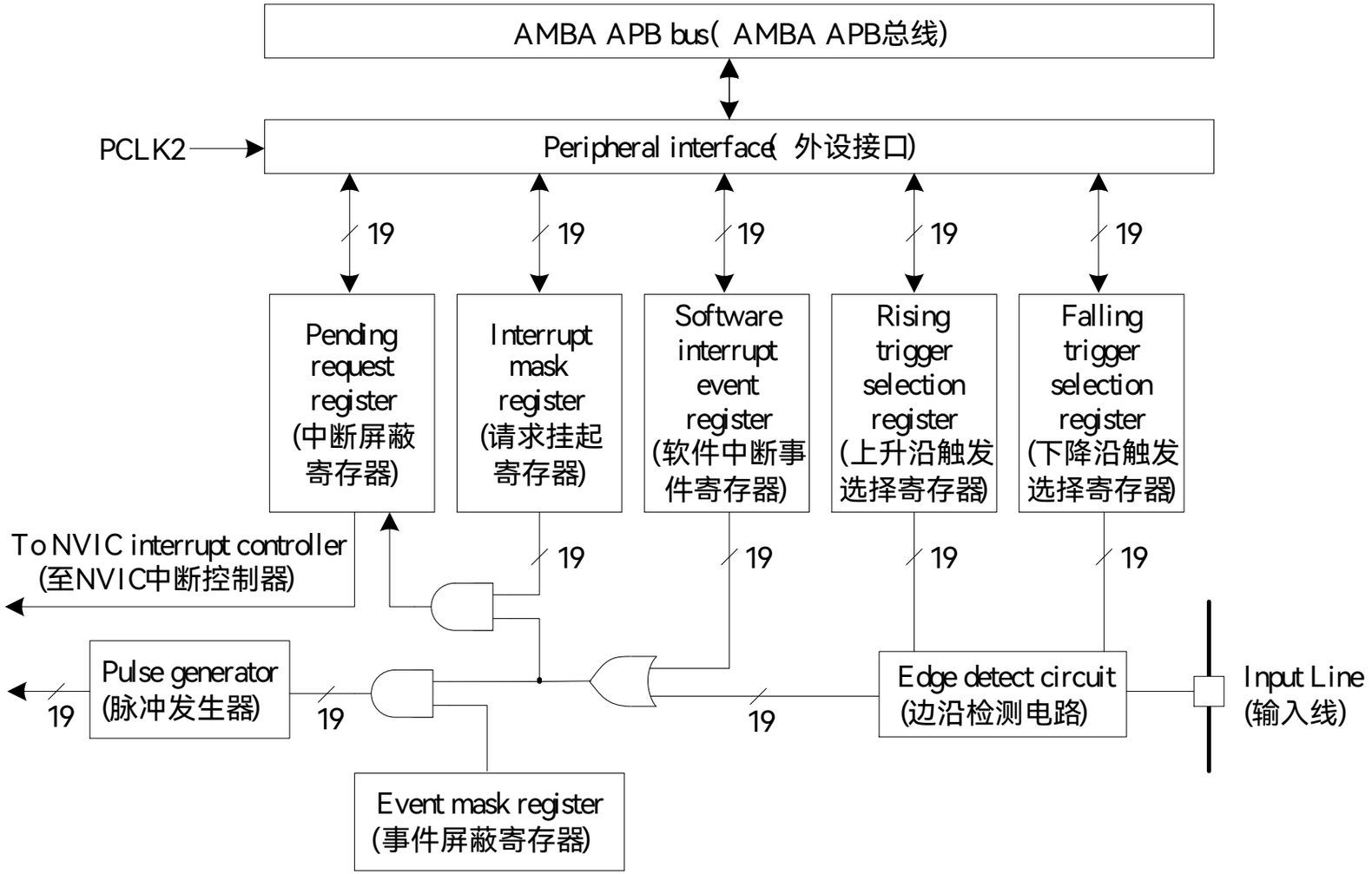
STM32芯片之外的外设的中断（I/O端口）由EXTI和NVIC共同负责，即STM32的每一个GPIO引脚都可以配置成一个外部中断触发源

EXTI（External interrupt/event controller，外部中断/事件控制器）支持19个外部中断/事件请求，每个中断/事件都有独立的触发和屏蔽设置，具有中断模式和事件模式两种设置模式。



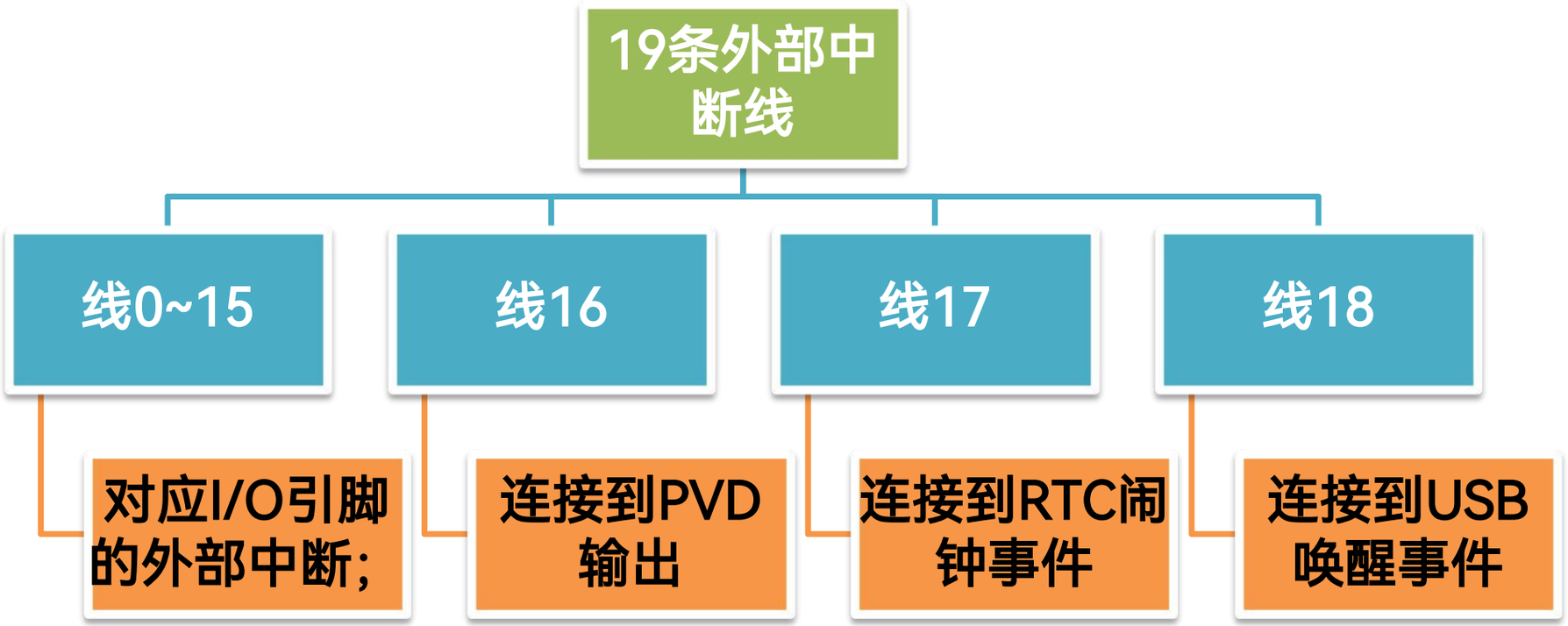
6.3 STM32外部中断EXTI

STM32 EXTI内部功能框图



6.3 STM32外部中断EXTI

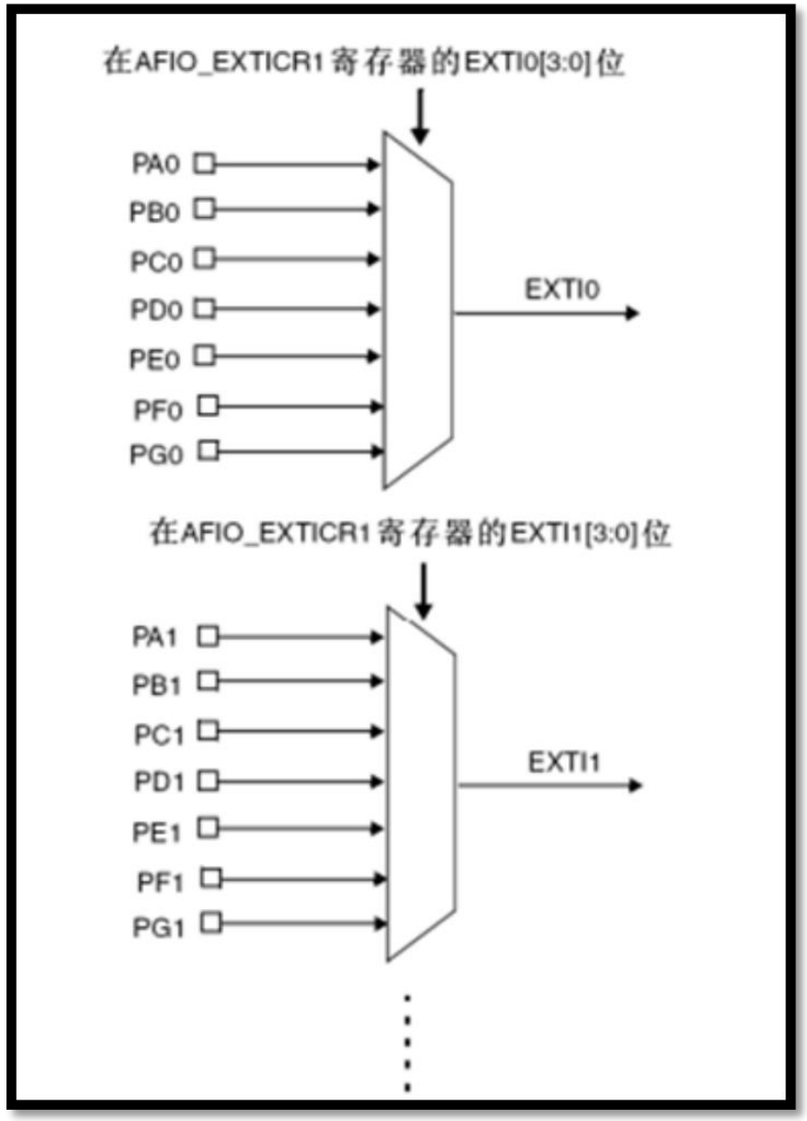
STM32中，每一个GPIO都可以触发一个外部中断



6.3 STM32外部中断EXTI

GPIO的中断是以组为单位的，同组的外部中断公用一条外部中断线。

例如：PA0、PB0、PC0、PD0、PE0、PF0、PG0这些为一组，如果使用PA0作为外部中断源，那么PB0、PC0、PD0、PE0、PF0、PG0就不能同时再作为外部中断使用了，在此情况下，只能使用类似于PB1、PC2这种末端序号不同的外部中断源。



GPIO引脚和外部中断线的映射关系图

6.3 STM32外部中断EXTI

中断标志	中断服务函数
EXTI0	void EXTI0_IRQHandler(void)
EXTI1	void EXTI1_IRQHandler(void)
EXTI2	void EXTI2_IRQHandler(void)
EXTI3	void EXTI3_IRQHandler(void)
EXTI4	void EXTI4_IRQHandler(void)
EXTI5-9	void EXTI9_5_IRQHandler(void)
EXTI10-15	void EXTI15_10_IRQHandler(void)

- EXTI0-EXTI4这5个外部中断有着各自独立的中断服务函数，如EXTI3的中断服务函数为

```
void EXTI3_IRQHandler(void);
```

- EXTI5-9共用一个中断服务函数

```
void EXTI9_5_IRQHandler(void);
```

- EXTI10-15共用一个中断服务函数

```
void
```

6.4 EXTI模块的标准外设库接口函数及应用



6.4.1 EXTI标准外设库接口函数



6.4.2 EXTI标准外设库中断配置步骤



6.4.3 EXTI标准外设库应用实例

6.4.1 EXTI标准外设库接口函数

EXTI标准外设库函数

标准外设库的stm32f10x_exti.c文件定义了EXTI相关的库函数。

stm32f10x_exti.h头文件中声明了EXTI相关的库函数，共有8种。

```
154 /** @defgroup EXTI_Exported_Functions
155   * @{
156   */
157
158 void EXTI_DeInit(void);
159 void EXTI_Init(EXTI_InitTypeDef* EXTI_InitStruct);
160 void EXTI_StructInit(EXTI_InitTypeDef* EXTI_InitStruct);
161 void EXTI_GenerateSWInterrupt(uint32_t EXTI_Line);
162 FlagStatus EXTI_GetFlagStatus(uint32_t EXTI_Line);
163 void EXTI_ClearFlag(uint32_t EXTI_Line);
164 ITStatus EXTI_GetITStatus(uint32_t EXTI_Line);
165 void EXTI_ClearITPendingBit(uint32_t EXTI_Line);
166
```

6.4.1 EXTI标准外设库接口函数



6.4.1 EXTI标准外设库接口函数

EXTI_InitTypeDef结构体

```
typedef struct
{
  uint32_t EXTI_Line;
     //中断/事件线
  EXTIMode_TypeDef EXTI_Mode;
     //EXTI模式
  EXTITrigger_TypeDef EXTI_Trigger;
     //触发类型
  FunctionalState EXTI_LineCmd;
     //EXTI使能
}EXTI_InitTypeDef;
```

指定某个具体的外部中断线路，如外部中断线0，EXTI_Line0

用于设置EXTI线路是中断请求模式还是事件模式，取值可以是EXTI_Mode_Interrupt或EXTI_Mode_Event

用于设置外部中断的触发方式，上升沿触发或下降沿触发等

用于使能或失能外部中断线路，取值为ENABLE或DISABLE

6.4.2 EXTI标准外设库中断配置步骤

1 GPIO初始化配置

```
GPIO_InitTypeDef
GPIO_InitStructure;
GPIO_InitStructure.GPIO_Pin =
GPIO_Pin_3;
GPIO_InitStructure.GPIO_Mode =
GPIO_Mode_IN_FLOATING;
    //初始化I/O口为浮空输入模式
```

2 开启I/O端口的时钟和复用时钟

GPIO用作EXTI外部中断功能引脚，必须开启复用功能，打开相应引脚的AFIO时钟，通过调用RCC_APB2PeriphClockCmd()函数实现：

```
RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOE|RCC_APB2Periph_AFIO,ENABLE);
```

6.4.2 EXTI标准外设库中断配置步骤

3 设置I/O引脚与中断线路的映射关系

GPIO引脚作为中断功能引脚用来触发外部中断时，需将GPIO的引脚与相应的中断线关联在一起，使用库函数GPIO_EXTILineConfig来实现这种映射关系。如

```
GPIO_EXTILineConfig(GPIO_PortSourceGPIOE,GPIO_PinSource3);
```

4 初始化EXTI，配置EXTI相关参数并使能

```
EXTI_InitStructure.EXTI_Line = EXTI_Line3; //设置外部中断线3中断
EXTI_InitStructure.EXTI_Mode = EXTI_Mode_Interrupt;
    //设置为中断模式
EXTI_InitStructure.EXTI_Trigger = EXTI_Trigger_Rising_Falling;
    //设置为下降沿触发
EXTI_InitStructure.EXTI_LineCmd = ENABLE; //使能该外部中断线
```

6.4.2 EXTI标准外设库中断配置步骤

5 初始化NVIC，配置NVIC参数并使能

NVIC相关的配置主要包括配置中断优先级的中断分组，确定各具体中断的抢占优先级和响应优先级的大小。

```
NVIC_InitTypeDef NVIC_InitStructure;
NVIC_PriorityGroupConfig(NVIC_PriorityGroup_2);
NVIC_InitStructure.NVIC_IRQChannel = EXTI3_IRQn;
NVIC_InitStructure.NVIC_IRQChannelPreemptionPriority = 0;
NVIC_InitStructure.NVIC_IRQChannelSubPriority = 1;
NVIC_InitStructure.NVIC_IRQChannelCmd = ENABLE;
NVIC_Init(&NVIC_InitStructure);
```

6.4.2 EXTI标准外设库中断配置步骤

6 编写中断服务程序

中断服务程序主要包括检测中断线路的状态、中断处理的内容和清除相关的中断。例如EXTI2的中断服务程序:

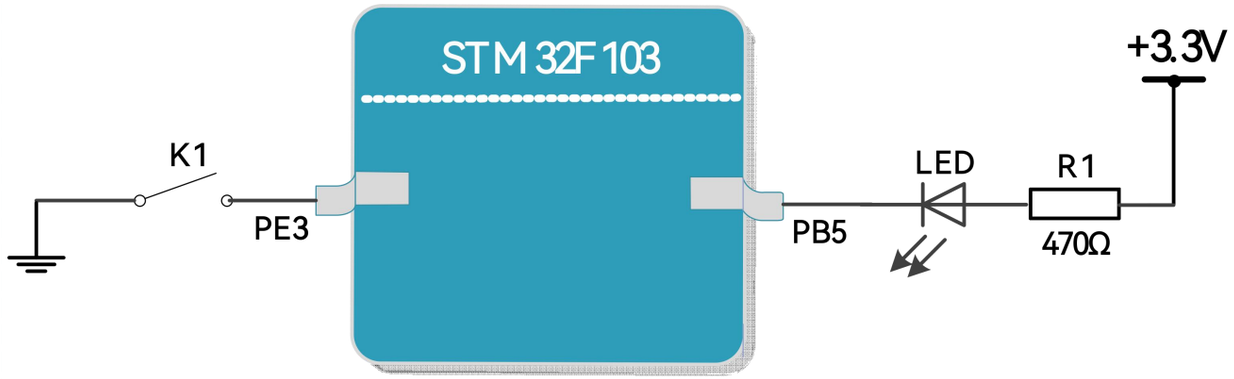
```
void EXTI2_IRQHandler(void)
{
   if(EXTI_GetITStatus(EXTI_Line2)!=RESET)
     //判断某个线上的中断是否发生
   {   中断逻辑…
       EXTI_ClearITPendingBit(EXTI_Line2); //清除中断标志
   }
}
```

6.4.3 EXTI标准外设库应用实例

功能

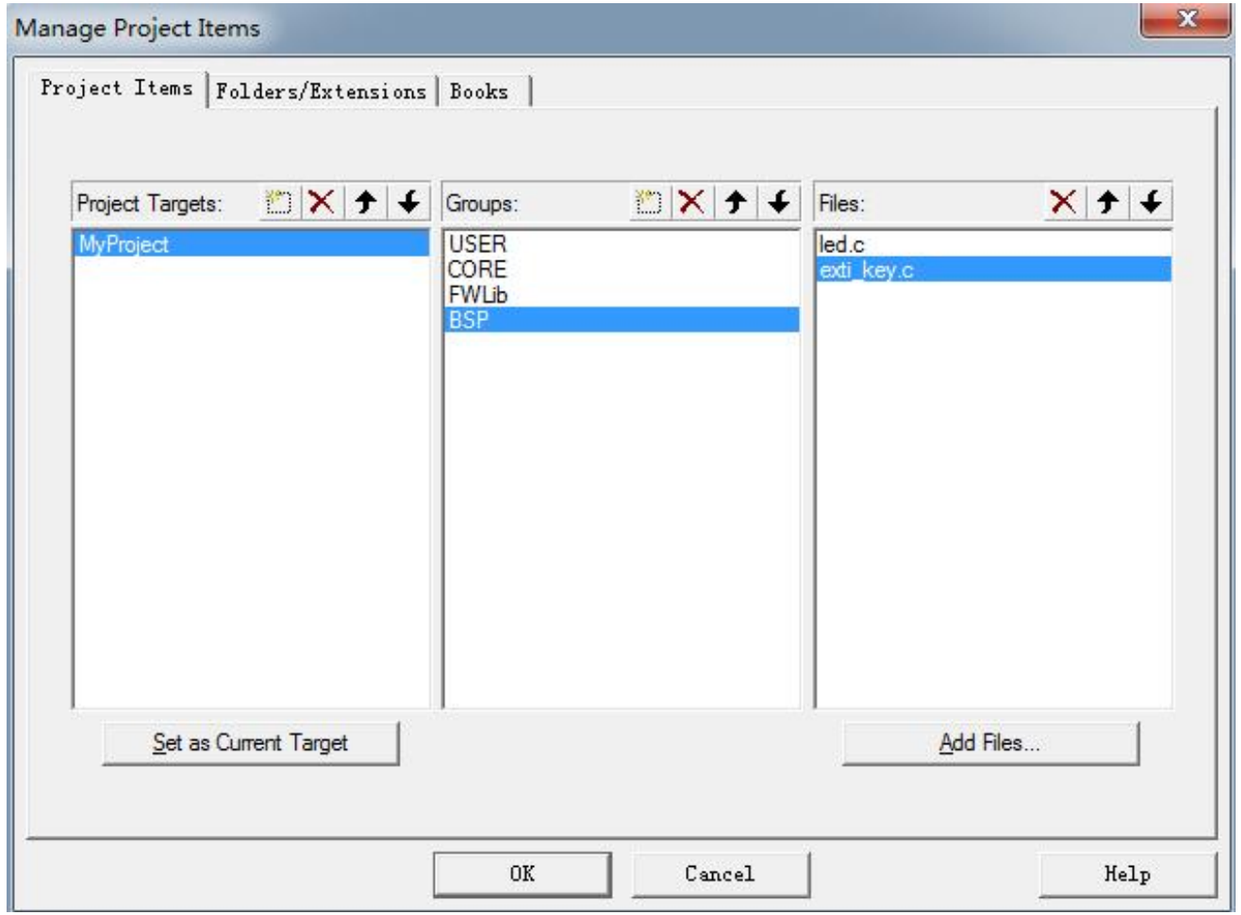
通过一个按键控制LED灯状态翻转，每按下按键一次，LED灯的状态变化一次。

硬件设计



LED连接在STM32F103的PB5引脚上，
按键K1连接在STM32F103的PE3引脚上

6.4.3 EXTI标准外设库应用实例



添加exti_key.c到工程中

软件设计

在BSP文件夹下新建exti_key.h和exti_key.c文件，并加入到工程中，添加方法为：在快捷功能栏中找到，单击进入，在“Groups”一栏中找到“BSP”，选中，然后单击“Add Files”，添加要加入工程的.c文件，具体操作如图所示。

6.4.3 EXTI标准外设库应用实例

1. 新建exti_key.h文件

```
#ifndef __EXTI_KEY_H
#define __EXTI_KEY_H
#include "stm32f10x.h"
void EXTI_Key_Init(void);
#endif
```

6.4.3 EXTI标准外设库应用实例

2．新建exti_key.c文件

```
#include "exti_key.h"
#include "misc.h"

void EXTI_Key_Init(void)
{
    GPIO_InitTypeDef GPIO_InitStructure;

RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOE|RCC_
APB2Periph_AFIO,ENABLE);
    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_3;
    GPIO_InitStructure.GPIO_Mode =
GPIO_Mode_IN_FLOATING;
    GPIO_Init(GPIOE,&GPIO_InitStructure);
    NVIC_InitTypeDef NVIC_InitStructure;
    NVIC_PriorityGroupConfig(NVIC_PriorityGroup_2);
    NVIC_InitStructure.NVIC_IRQChannel = EXTI3_IRQn;

NVIC_InitStructure.NVIC_IRQChannelPreemptionPriority =
    NVIC_InitStructure.NVIC_IRQChannelSubPriority = 1;
    NVIC_InitStructure.NVIC_IRQChannelCmd = ENABLE;
    NVIC_Init(&NVIC_InitStructure);
    EXTI_InitTypeDef EXTI_InitStructure;
    EXTI_ClearITPendingBit(EXTI_Line3);

GPIO_EXTILineConfig(GPIO_PortSourceGPIOE,GPIO_PinSou
rce3);
    EXTI_InitStructure.EXTI_Line = EXTI_Line3;
    EXTI_InitStructure.EXTI_Mode = EXTI_Mode_Interrupt;
    EXTI_InitStructure.EXTI_Trigger = EXTI_Trigger_Falling;
    EXTI_InitStructure.EXTI_LineCmd = ENABLE;
    EXTI_Init(&EXTI_InitStructure);
}
```

6.4.3 EXTI标准外设库应用实例

3. main.c文件

```
#include "stm32f10x.h"

#include "led.h"
#include "exti_key.h"

int main(void)
{
    LED_Init();
    GPIO_ResetBits(GPIOB,GPIO_Pin_5);
EXTI_Key_Init();
    while(1)
    {
    }
  }
```

```
void EXTI3_IRQHandler(void)
{
    if(EXTI_GetITStatus(EXTI_Line3) != RESET)
    {

GPIO_WriteBit(GPIOB,GPIO_Pin_5,(BitAction)((1-
GPIO_ReadOutputDataBit(GPIOB,GPIO_Pin_5))));
        EXTI_ClearITPendingBit(EXTI_Line3);
    }
}
```

6.4.3 EXTI标准外设库应用实例

另外一种中断处理程序，实现LED灯翻转

```
uint8_t led =1;
void EXTI3_IRQHandler(void)
{
    if(EXTI_GetITStatus(EXTI_Line3) != RESET)
    {
        led = ~led; //状态翻转
        //如果等于1，则PB5复位点亮，否则置1熄灭
      if(led  == 1)
        GPIO_ResetBits(GPIOB,GPIO_Pin_5);
      else
        GPIO_SetBits(GPIOB,GPIO_Pin_5);
    }
    EXTI_ClearITPendingBit(EXTI_Line3); //清除EXTI3的中断标
志位
}
```





6.4.3 EXTI标准外设库应用实例

使用STM32的外部中断步骤

使用RCC_APB2PeriphClockCmd()函数配置时钟，初始化GPIO引脚为复用功能

使用GPIO_EXTILineConfig()函数配置I/O引脚与外部中断线路的映射关系

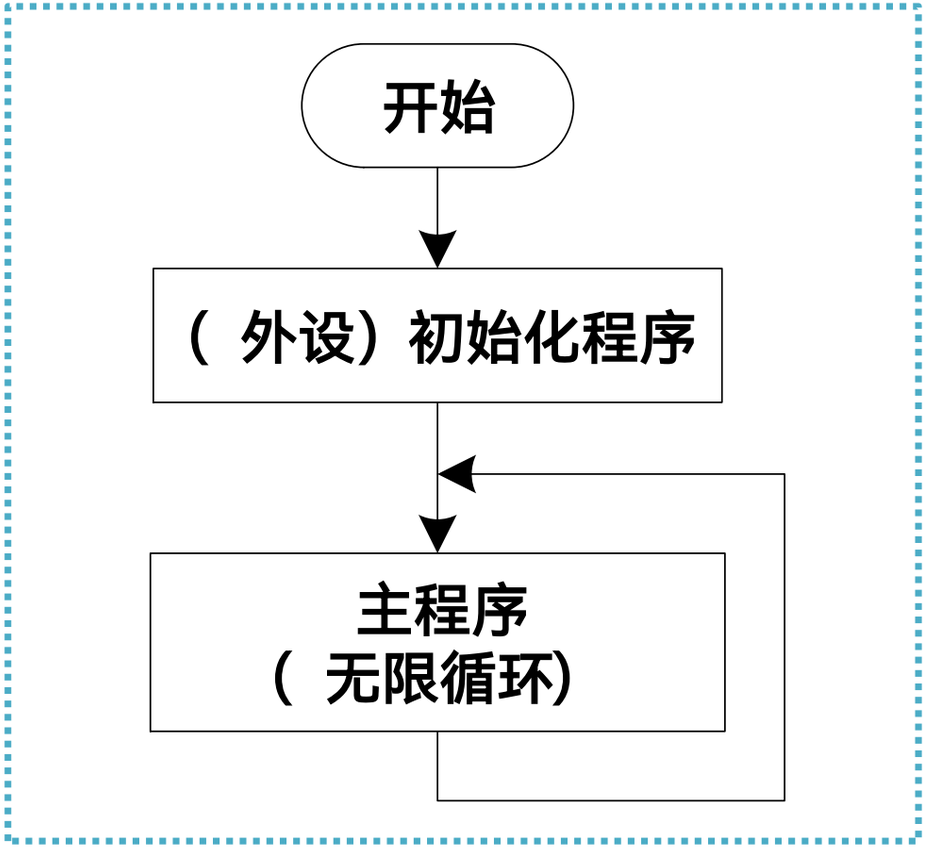
使用EXTI_Init()函数配置外部中断

使用NVIC_Init()配置中断优先级等

编写中断服务函数

6.4.3 EXTI标准外设库应用实例

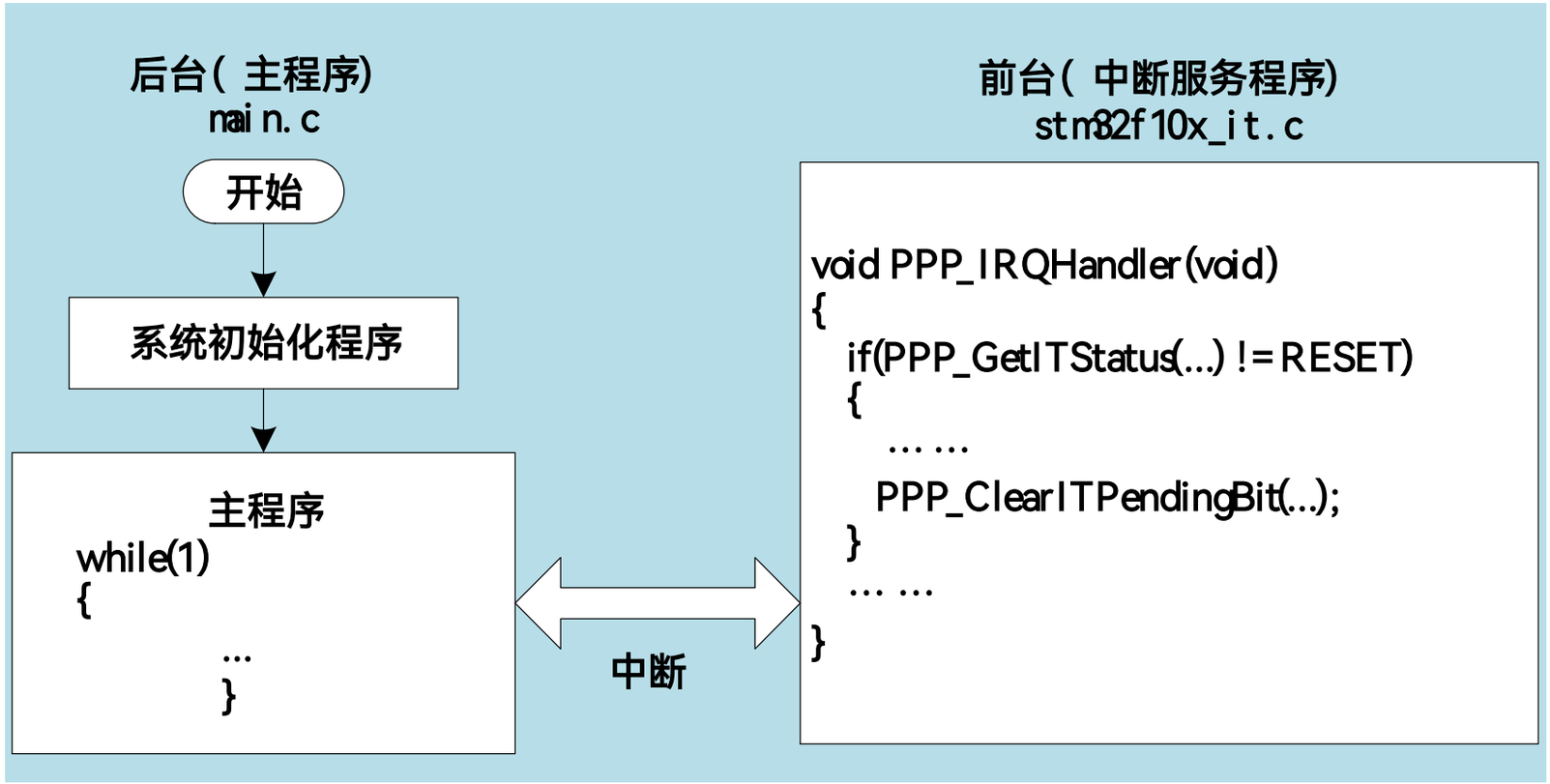
嵌入式系统开发的程序架构由初始化部分和主程序部分组成，初始化部分涉及到系统所用外设的初始化配置程序，主程序部分一般是由while(1){ }构成的无限循环，程序实体一直循环不停地轮流执行



结构简单，容易实现，但由于主程序部分是一个轮流执行的无限循环体，遇到突发事件，无法保证系统响应的及时性

6.4.3 EXTI标准外设库应用实例

前后台系统的程序架构图



前台程序通过中断来处理事件，后台程序是一个无限循环的应用程序实体，负责嵌入式系统软、硬件资源的分配、管理等日常事务。

6.5 EXTI模块的HAL库接口函数及应用

目 录


6.5.1 EXTI的 HAL库接口函数


6.5.2 EXTI的HAL库应用实例

6.5.1 EXTI的HAL库接口函数

EXTI的HAL库接口函数的源码定义在stm32f1xx_hal_exti.c源文件中，其对应的头文件stm32f1xx_hal_exti.h声明了EXTI所有的库函数

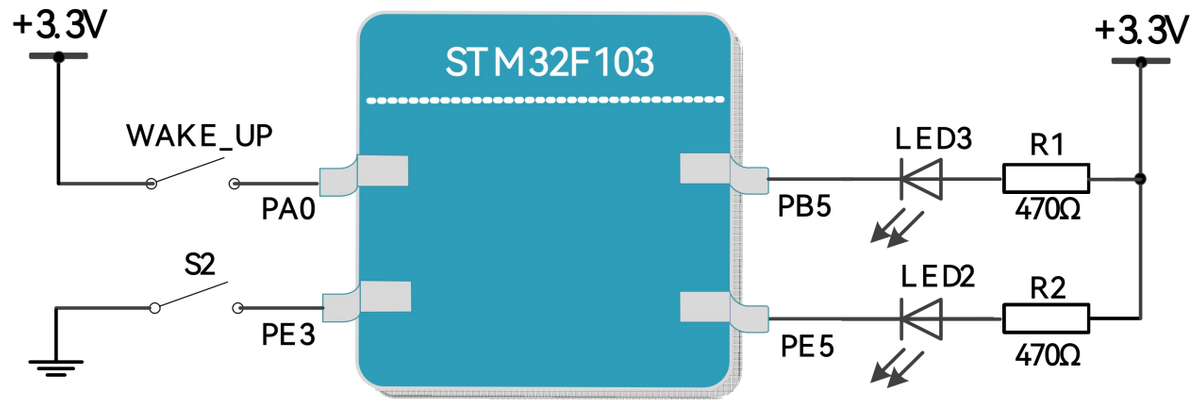
类型	函数及功能描述
配置函数	HAL_EXTI_SetConfigLine(EXTI_HandleTypeDef *hexti, EXTI_ConfigTypeDef *pExtiConfig); 功能描述：设置外部中断EXTI的外部中断线路
	HAL_EXTI_GetConfigLine(EXTI_HandleTypeDef *hexti, EXTI_ConfigTypeDef *pExtiConfig); 功能描述：获取已配置好的外部中断EXTI的外部中断线路
	HAL_StatusTypeDef HAL_EXTI_ClearConfigLine(EXTI_HandleTypeDef *hexti); 功能描述：将已设置好的外部中断线路清除
	HAL_EXTI_GetHandle(EXTI_HandleTypeDef *hexti, uint32_t ExtiLine); 功能描述：获取EXTI的句柄
引脚功能操作函数	void HAL_EXTI_IRQHandler(EXTI_HandleTypeDef *hexti); 功能描述：外部中断EXTI的中断处理函数
	uint32_t HAL_EXTI_GetPending(EXTI_HandleTypeDef *hexti, uint32_t Edge); 功能描述：获取被挂起的外部中断线/事件线
	void HAL_EXTI_ClearPending(EXTI_HandleTypeDef *hexti, uint32_t Edge); 功能描述：清除被挂起的外部中断/事件线
	void HAL_EXTI_GenerateSWI(EXTI_HandleTypeDef *hexti); 功能描述：产生一个软件中断

6.5.2 EXTI的HAL库应用实例

功能

采用基于HAL库设计方式，通过两个按键分别控制两个LED灯状态翻转。

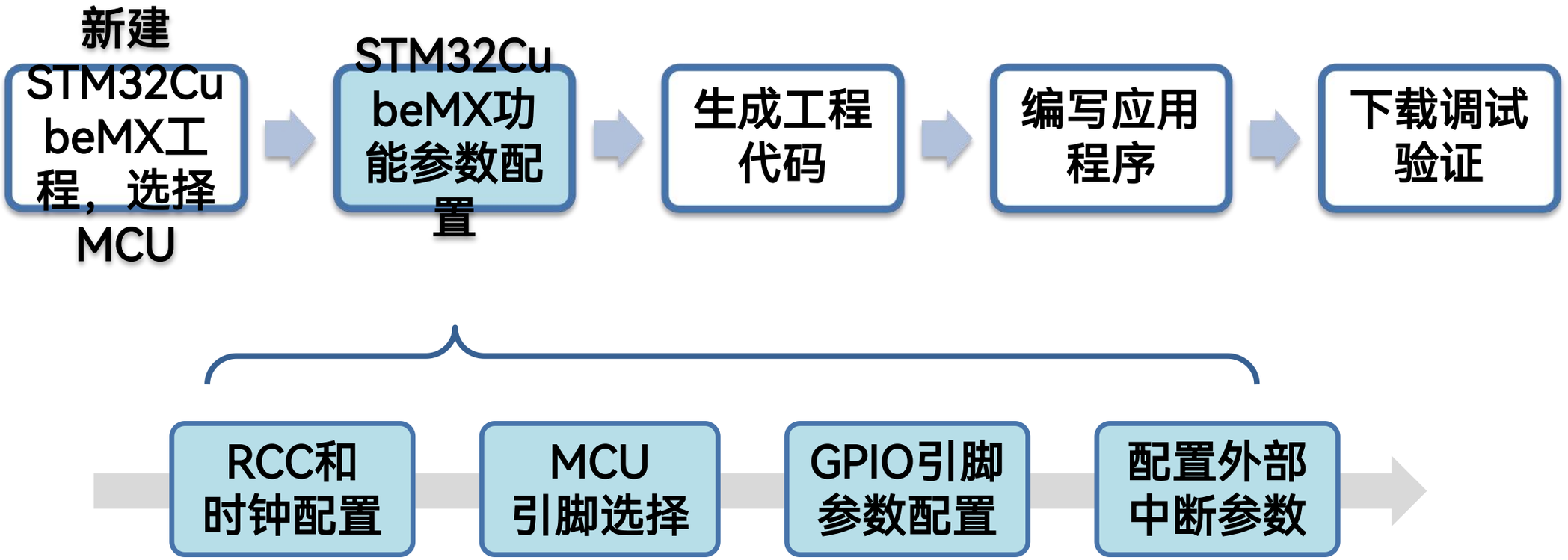
硬件设计



按键WAKE_UP连接在STM32F103的PA0引脚上，按下时PA0为高电平，按键S2连接在STM32F103的PE3引脚上。

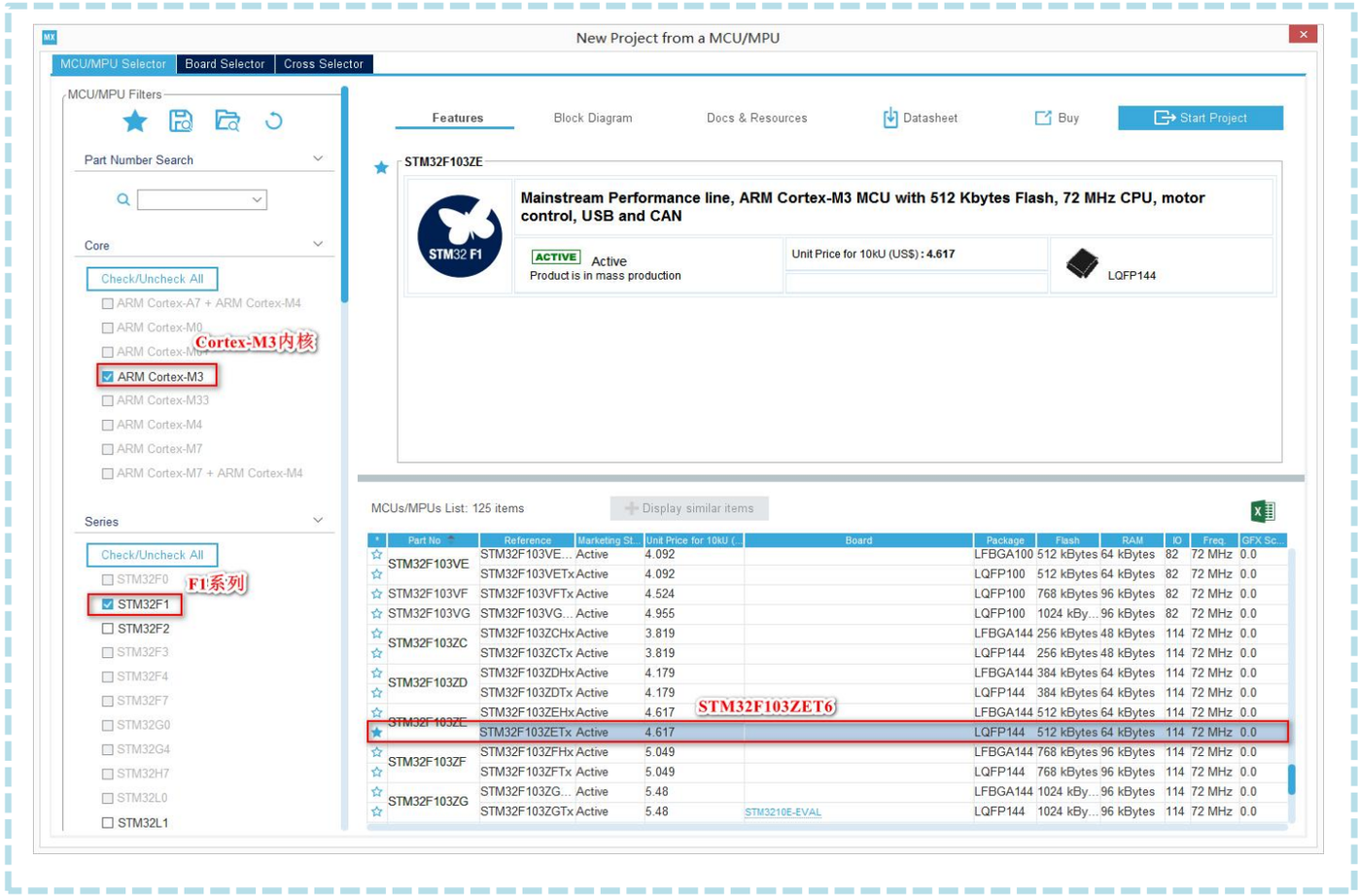
6.5.2 EXTI的HAL库应用实例

软件设计流程



软件设计——新建STM32CubeMX工程，选择MCU

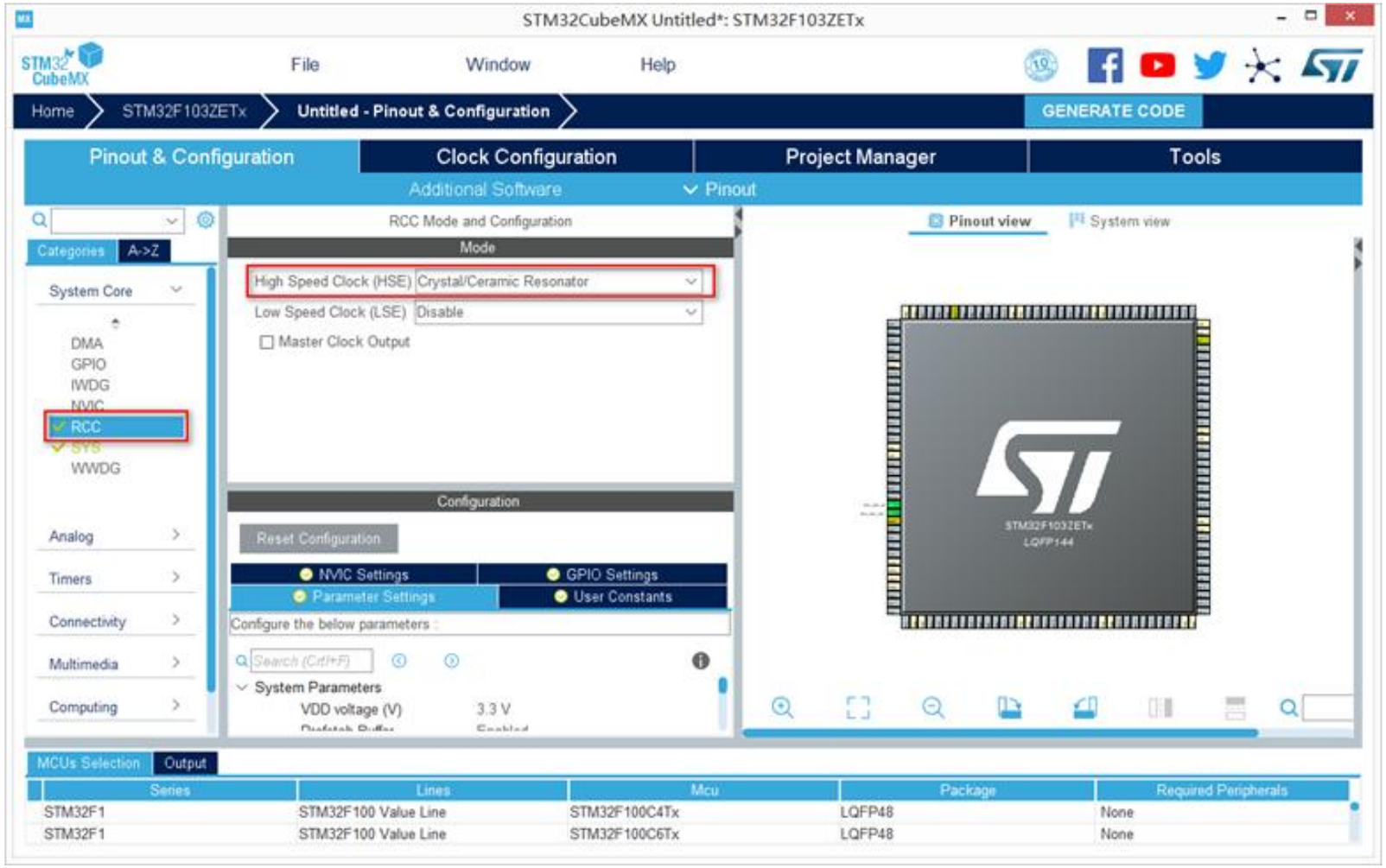
在D盘或其他盘符目录下新建一个文件夹，用来存放后面建立的STM32CubeMX工程。需要注意的是：所建工程文件名最好是英文名称，且最好是英文路径。这里采用STM32F103ZET6芯片。



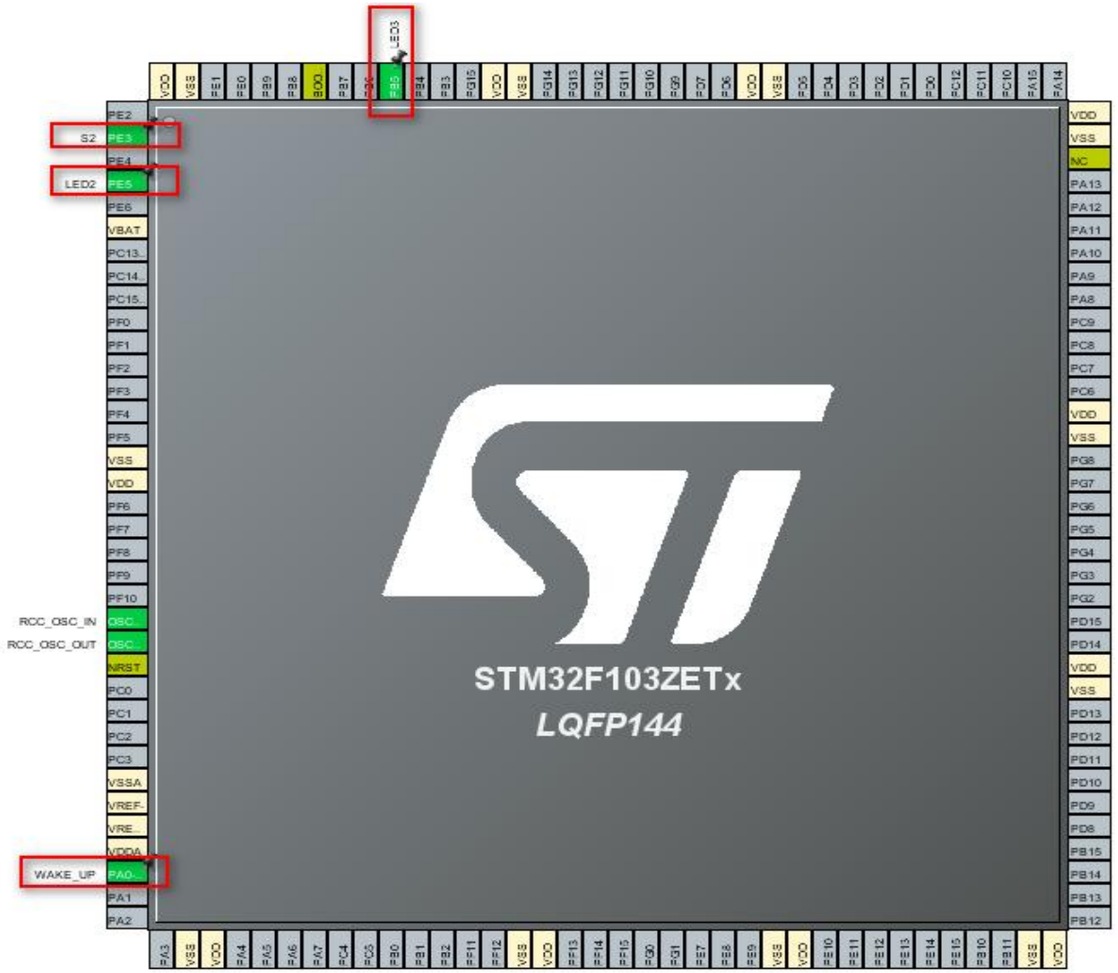
软件设计——STM32CubeMX功能参数配置

RCC配置

HSE选择“Crystal/Ceramic Resonator”(晶振/陶瓷谐振器），LSE选择“Disable”



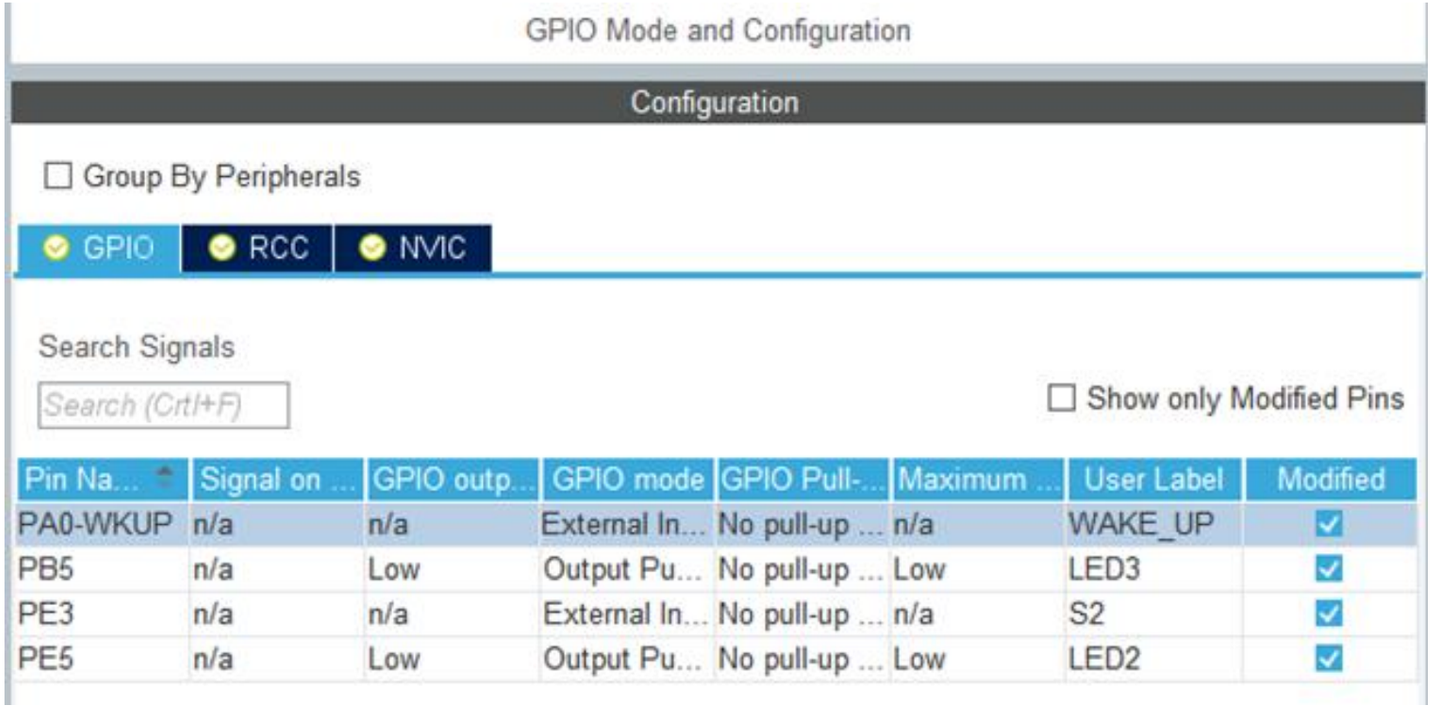
软件设计——STM32CubeMX功能参数配置



MCU引脚选择

- LED2连接在PE5引脚上，LED3连接在PB5引脚上，设置为GPIO_Output；
- 设置PA0为GPIO_EXTI0外部中断线作为WAKE_UP按键使用，用于控制LED3灯闪烁；
- 设置PE3为GPIO_EXTI3作为按键S2使用，用于控制LED2灯闪烁。

软件设计——STM32CubeMX功能参数配置



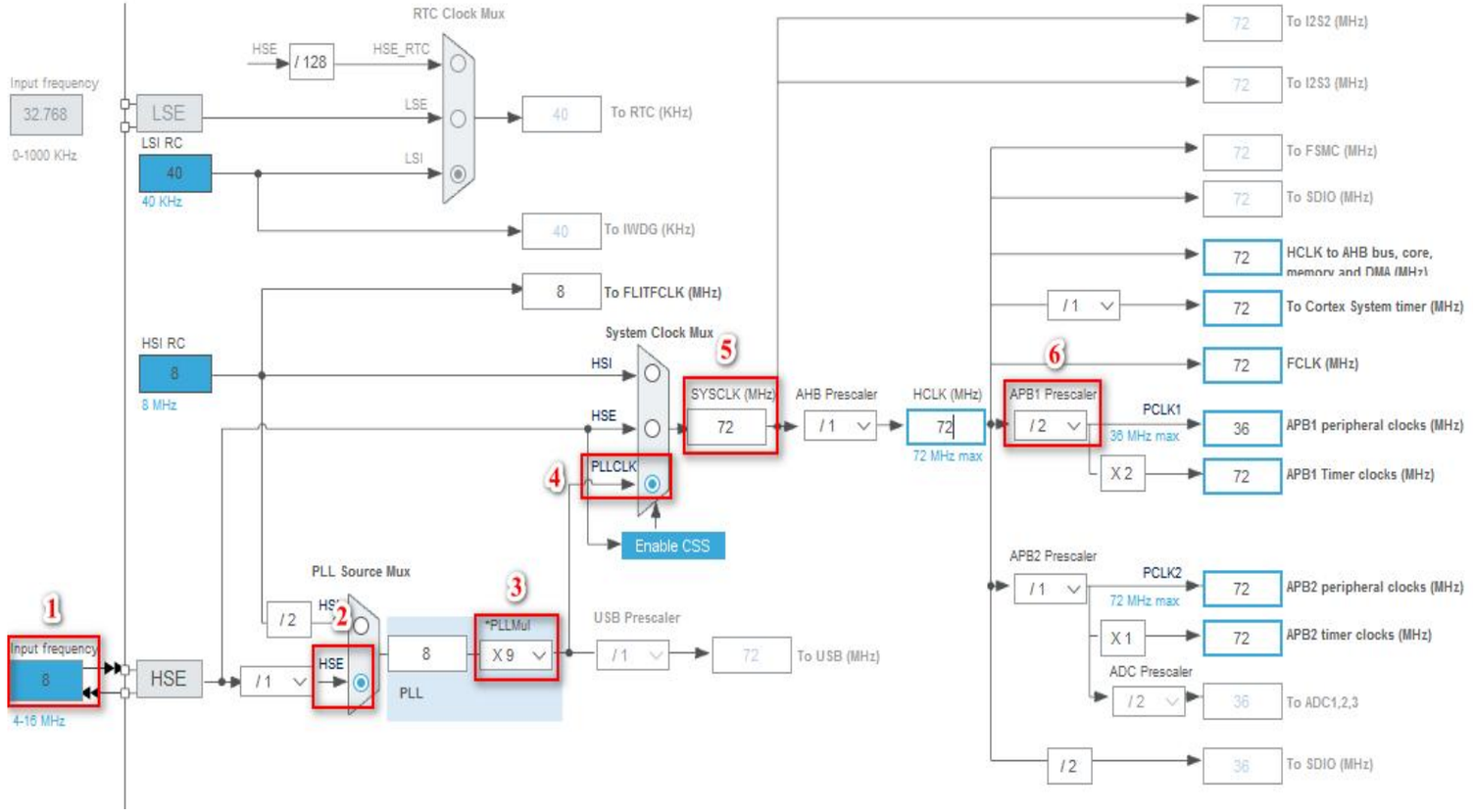
Pin Na...	Signal on ...	GPIO outp...	GPIO mode	GPIO Pull-...	Maximum ...	User Label	Modified
PA0-WKUP	n/a	n/a	External In...	No pull-up ...	n/a	WAKE_UP	☑
PB5	n/a	Low	Output Pu...	No pull-up ...	Low	LED3	☑
PE3	n/a	n/a	External In...	No pull-up ...	n/a	S2	☑
PE5	n/a	Low	Output Pu...	No pull-up ...	Low	LED2	☑

GPIO引脚参数配置

- 在“GPIO”参数配置中设置PA0为上升沿触发；
- PE3的“GPIO mode”设置为下降沿触发，添加用户标签S2。
- PB5和PE5两个LED灯设置为推挽输出，分别添加用户标签为LED2和LED3。

软件设计——STM32CubeMX功能参数配置

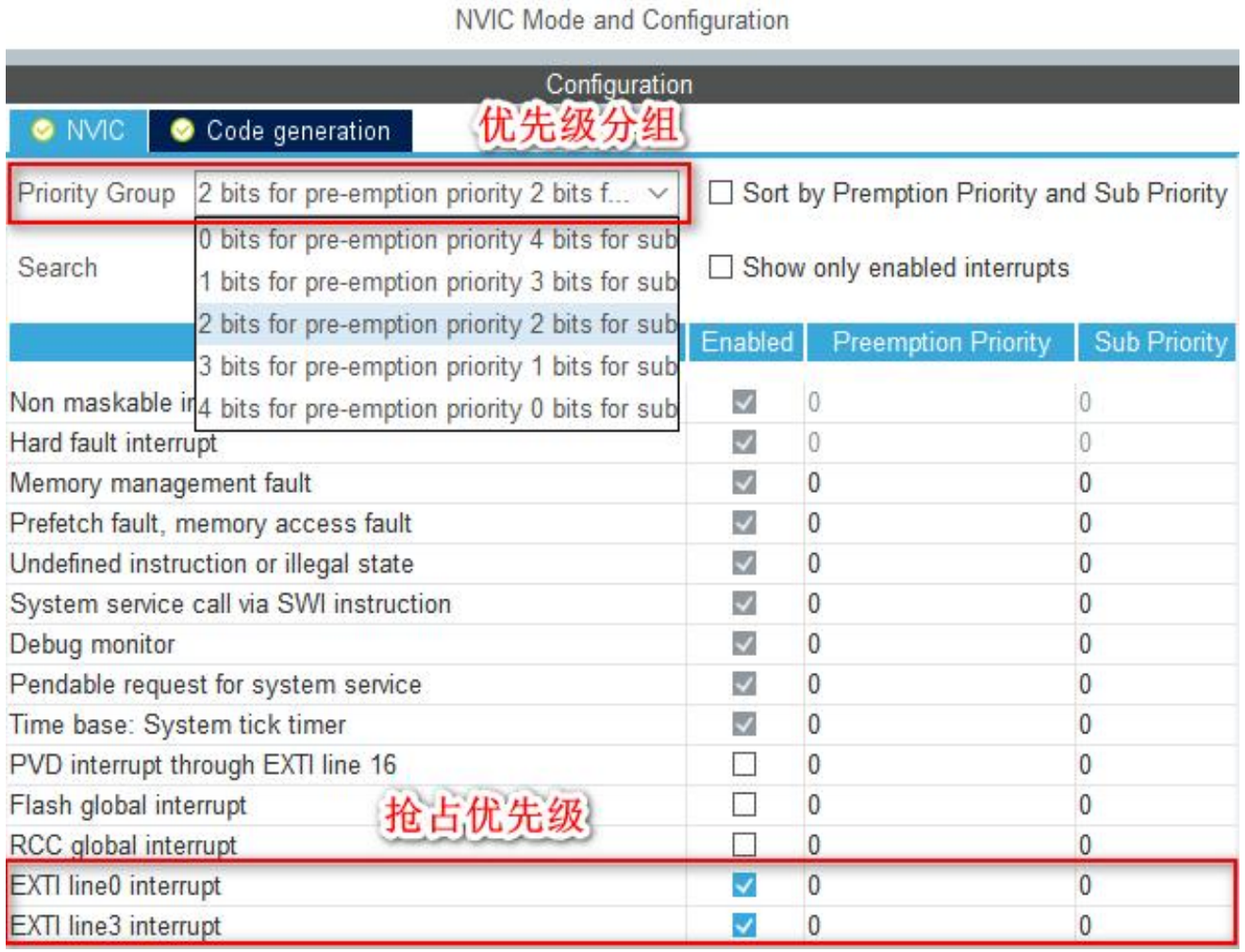
时钟配置



通过图形化方式直观对系统时钟进行配置，系统时钟采用外部高速时钟，配置STM32F103系列芯片最大时钟为72MHz，配置APB2为72MHz，配置APB1为36MHz。

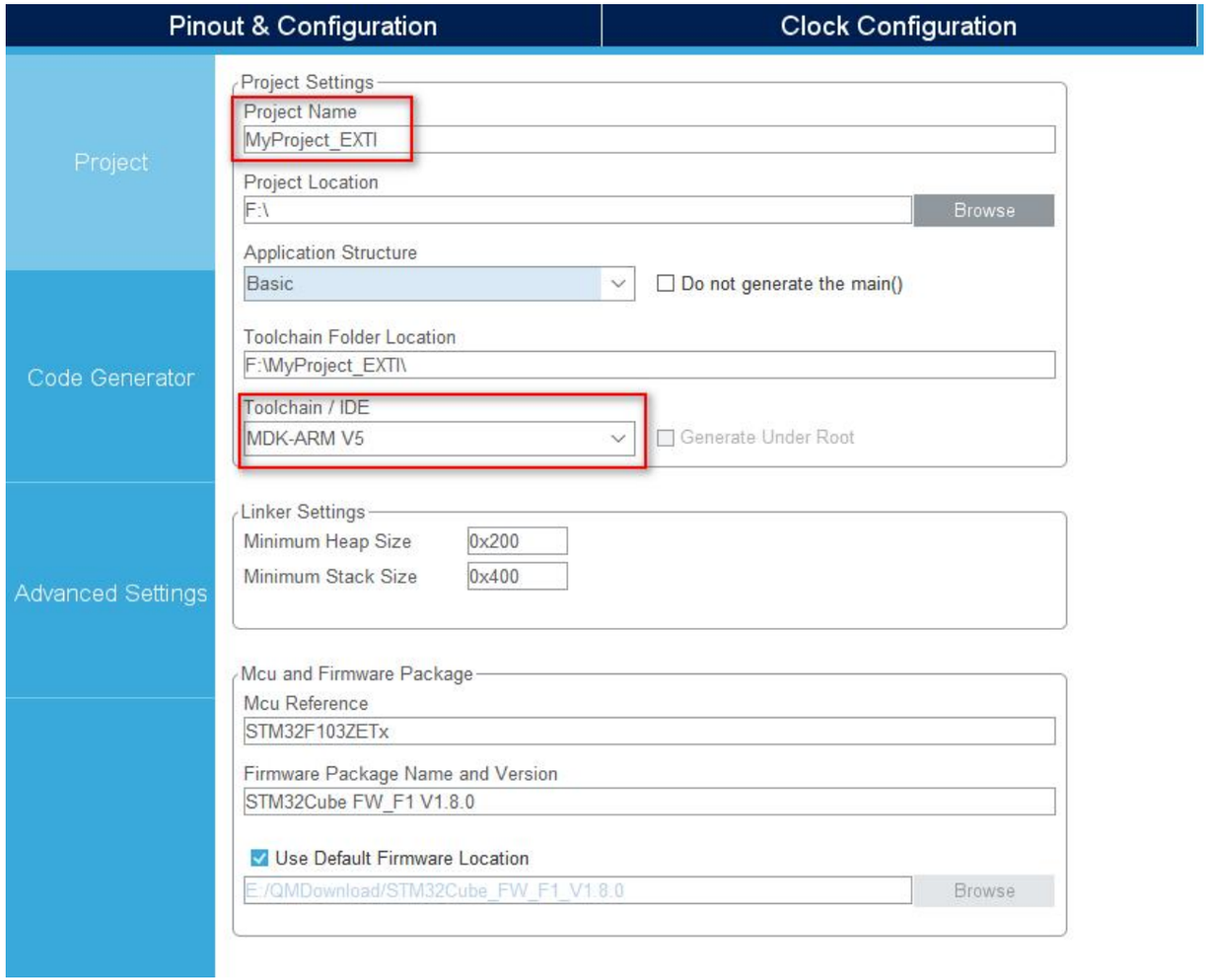
软件设计——STM32CubeMX功能参数配置

配置外部中断参数



- 在“NVIC”选项页中，设置“Priority Group”进行优先级分组；
- 勾选“EXIT_Line0 interrupt”使能PA0外部中断线；
- 类似地，勾选“EXTI_Line3 interrupt”使能PE3外部中断线。

软件设计——生成工程代码

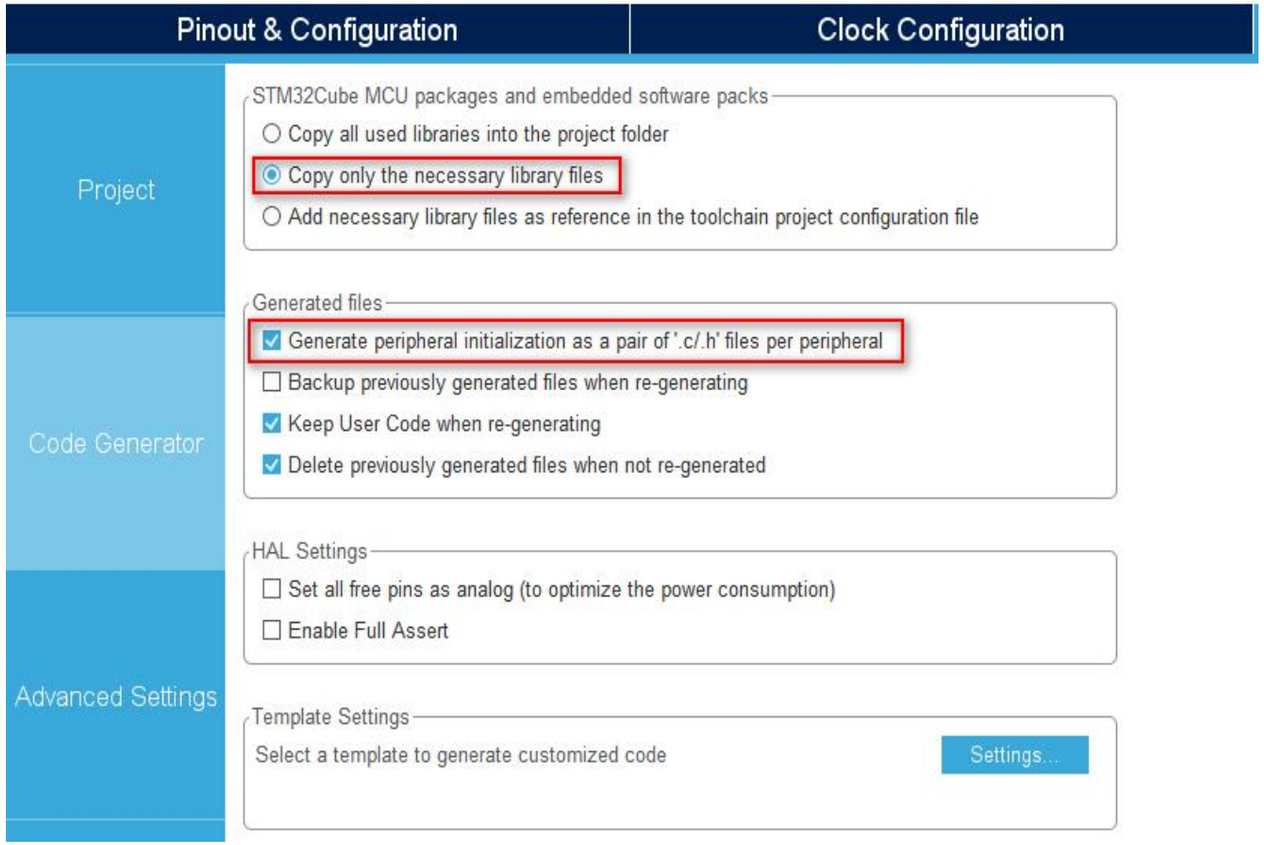


配置keil工程名称和存放位置

定义“Project Name”为“MyProject_EXTI”，“Toolchain/IDE”选择“MDK-ARM V5”；



软件设计——生成工程代码



在“Code Generator”选项栏中找到“Generated files”框，勾选“Generate peripheral initialization as a pair of ’.c/.h’ files per IP”，将外设初始化的代码设置生成为独立的.c源文件和.h头文件。

软件设计——生成工程代码

通过STM32CubeMX的菜单栏中的“Generate Code”生成工程代码，生成代码后，会提示是否打开该工程窗口



软件设计——编写应用程序

- 在keil 中单击编译按钮，对生成的工程进行编译；
- 在stm32f1xx_it.c文件中可以找到所定义的中断函数；
- 外部的两个按键中断触发后，首先会调用GPIO相应引脚的外部中断处理函数。

main.c | gpio.c | stm32f1xx_it.h | **stm32f1xx_it.c** | stm32f1xx_hal_gpio.c

```
188  /* STM32F1xx Peripheral Interrupt Handlers
189  /* Add here the Interrupt Handlers for the used peripherals.
190  /* For the available peripheral interrupt handler names,
191  /* please refer to the startup file (startup_stm32f1xx.s).
192  /*****************************************************************
193
194  /**
195  * @brief This function handles EXTI line0 interrupt.
196  */
197  void EXTI0_IRQHandler(void)
198  {
199    /* USER CODE BEGIN EXTI0_IRQn 0 */
200
201    /* USER CODE END EXTI0_IRQn 0 */
202    HAL_GPIO_EXTI_IRQHandler(GPIO_PIN_0);
203    /* USER CODE BEGIN EXTI0_IRQn 1 */
204
205    /* USER CODE END EXTI0_IRQn 1 */
206  }
207
208  /**
209  * @brief This function handles EXTI line3 interrupt.
210  */
211  void EXTI3_IRQHandler(void)
212  {
213    /* USER CODE BEGIN EXTI3_IRQn 0 */
214
215    /* USER CODE END EXTI3_IRQn 0 */
216    HAL_GPIO_EXTI_IRQHandler(GPIO_PIN_3);
217    /* USER CODE BEGIN EXTI3_IRQn 1 */
218
219    /* USER CODE END EXTI3_IRQn 1 */
220  }
221
222  /* USER CODE BEGIN 1 */
223
```

外部中断线0的中断处理函数

外部中断线3的中断处理函数

软件设计——编写应用程序

```
void HAL_GPIO_EXTI_IRQHandler(uint16_t GPIO_Pin)
{
  /* EXTI line interrupt detected */
  if (__HAL_GPIO_EXTI_GET_IT(GPIO_Pin) != RESET)
  {
    __HAL_GPIO_EXTI_CLEAR_IT(GPIO_Pin);
    HAL_GPIO_EXTI_Callback(GPIO_Pin);
  }
}
```

HAL_GPIO_EXTI_IRQHandler()函数原型

- 比如用户按键S2连接在PE3引脚上会触发外部中断EXTI3的中断处理程序EXTI3_IRQHandler(void)，而这个函数又调用HAL_GPIO_EXTI_IRQHandler(GPIO_PIN_3)函数，参数为GPIO_PIN_3，即EXTI3中断。单击右键跳转到这个函数，可以看到该函数的原型，

软件设计——编写应用程序

```
/* USER CODE BEGIN 4 */
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
   if(GPIO_Pin == GPIO_PIN_3)
   {
      HAL_GPIO_TogglePin(GPIOE,GPIO_PIN_5);
   }
  if(GPIO_Pin == GPIO_PIN_0)
  {
      HAL_GPIO_TogglePin(LED3_GPIO_Port,LED3_Pin);
  }
}
/* USER CODE END 4 */
```

- 当中断发生时，会调用HAL_GPIO_EXTI_Callback()函数，但程序中只给出了一个虚函数，需要用户添加应用程序代码来实现相关功能。
- 在main.c文件中的/*USER CODE BEGIN 4*/和/*USER CODE END 4*/之间添加中断回调函数相应的程序代码。

软件设计——下载调试验证

应用程序编写完毕

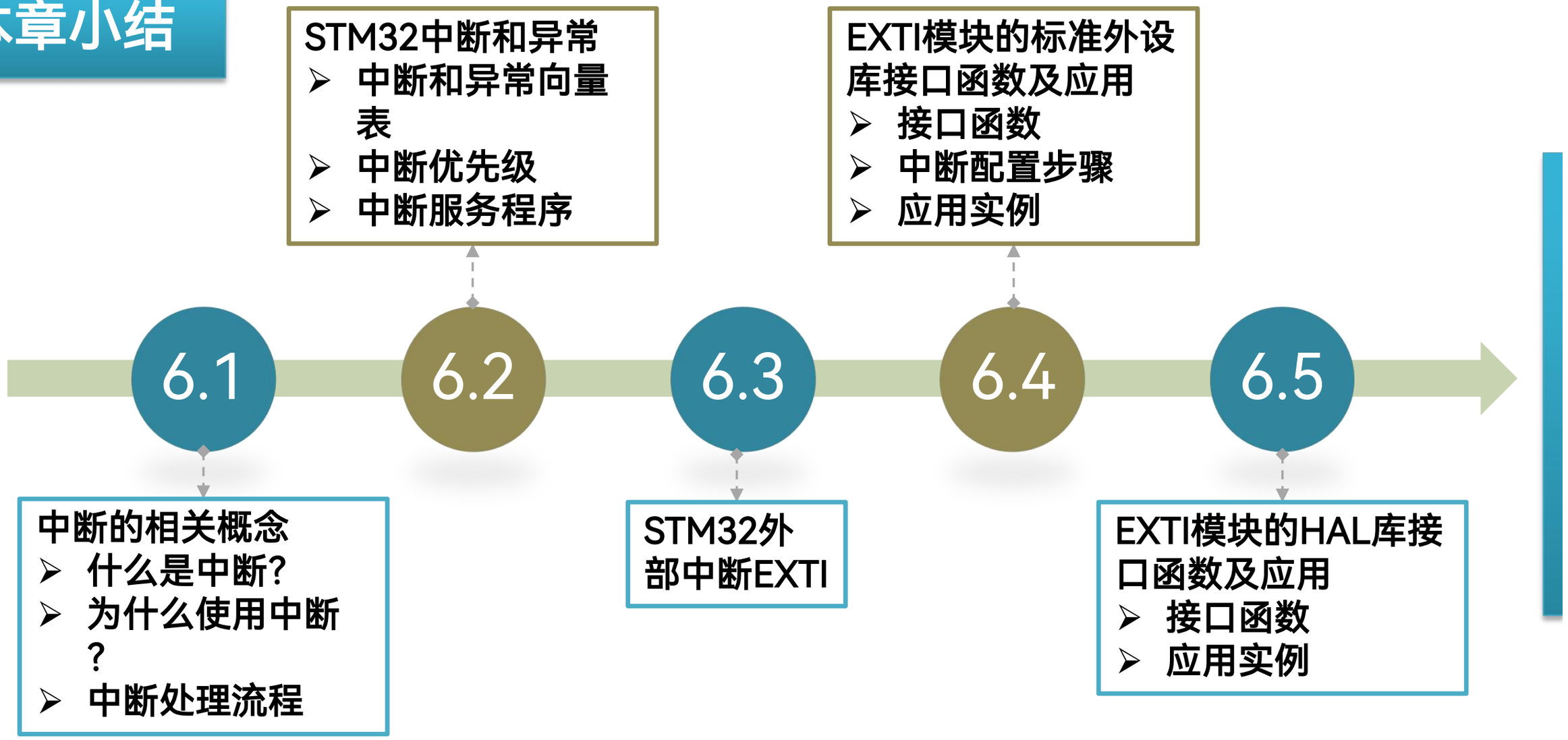
重新编译一下工程，生成工程hex文件

通过ST-LINK仿真器或其它下载器下载到目标板

进行验证

可以看到，按下相应的按键，其对应的LED灯的状态会发生改变。

本章小结


6.1
6.2
6.3
6.4
6.5
中断的相关概念
➢ 什么是中断?
➢ 为什么使用中断?
➢ 中断处理流程
STM32中断和异常
➢ 中断和异常向量表
➢ 中断优先级
➢ 中断服务程序
STM32外部中断EXTI
EXTI模块的标准外设库接口函数及应用
➢ 接口函数
➢ 中断配置步骤
➢ 应用实例
EXTI模块的HAL库接口函数及应用
➢ 接口函数
➢ 应用实例



The End!

